Web Technologies

.....AND SECURITY

Stones Dalitso Chindipha

Chapter 8

Introduction to Server Side Development with PHP

Objectives

1. Server Side Development
2. Client vs Server Roles
3. Web Server's Responsibility
4. Quick Tour of PHP
5. PHP Control Structures
6. PHP Functions

PHP Data Types

- Variables can store data of different types, and different data types can do different things.
- PHP supports the following data types:
  - String
  - Integer
  - Float (floating point numbers - also called double)
  - Boolean
  - Array
  - Object
  - NULL
  - Resource

The PHP var_dump() function returns the data type and value

```
<?php
$x = 5985;
var_dump($x);
?>
```

Data Type	Description
`Boolean`	A logical true or false value
`Integer`	Whole numbers
`Float`	Decimal numbers
`String`	Letters
`Array`	A collection of data of any type (covered in the next chapter)
`Object`	Instances of classes

TABLE 8.1 PHP Data Types

PHP Constants

- Constants are like variables except that once they are defined they cannot be changed or undefined.
- A constant is an identifier (name) for a simple value.
  - The value cannot be changed during the script.
- A valid constant name starts with a letter or underscore (no $ sign before the constant name).
- **Note: Unlike variables, constants are automatically global across the entire script.**
- To create a constant, use the define() function.
- **Syntax: define(name, value, case-insensitive)**
  - **Name:** Specifies the name of the constant
  - **Value:** Specifies the value of the constant
  - **Case-insensitive:** Specifies whether the constant name should be case-insensitive. Default is false

```php
<?php

# uppercase for constants is a programming convention
define("DATABASE_LOCAL", "localhost");
define("DATABASE_NAME", "ArtStore");
define("DATABASE_USER", "Fred");
define("DATABASE_PASSWD", "F5^7%ad");
...
# notice that no $ prefaces constant names
$db = new mysqli(DATABASE_LOCAL, DATABASE_NAME, DATABASE_USER,
  DATABASE_NAME);

?>
```

LISTING 8.4 PHP constants

PHP Operators

- Operators are used to perform operations on variables and values.
- PHP divides the operators in the following groups:
  - Arithmetic operators
  - Assignment operators
  - Comparison operators
  - Increment/Decrement operators
  - Logical operators
  - String operators
  - Array operators
  - Conditional assignment operators

PHP Arithmetic Operators

Operator	Name	Example	Result
+	Addition	$x + $y	Sum of $x and $y
-	Subtraction	$x - $y	Difference of $x and $y
*	Multiplication	$x * $y	Product of $x and $y
/	Division	$x / $y	Quotient of $x and $y
%	Modulus	$x % $y	Remainder of $x divided by $y
**	Exponentiation	$x ** $y	Result of raising $x to the $y'th power

PHP Comparison Operators

Operator	Name	Example	Result
==	Equal	$x == $y	Returns true if $x is equal to $y
===	Identical	$x === $y	Returns true if $x is equal to $y, and they are of the same type
!=	Not equal	$x != $y	Returns true if $x is not equal to $y
<>	Not equal	$x <> $y	Returns true if $x is not equal to $y
!==	Not identical	$x !== $y	Returns true if $x is not equal to $y, or they are not of the same type
>	Greater than	$x > $y	Returns true if $x is greater than $y
<	Less than	$x < $y	Returns true if $x is less than $y
>=	Greater than or equal to	$x >= $y	Returns true if $x is greater than or equal to $y
<=	Less than or equal to	$x <= $y	Returns true if $x is less than or equal to $y
<=>	Spaceship	$x <=> $y	Returns an integer less than, equal to, or greater than zero, depending on if $x is less than, equal to, or greater than $y. Introduced in PHP 7.

PHP Logical Operators

Operator	Name	Example	Result
and	And	$x and $y	True if both $x and $y are true
or	Or	$x or $y	True if either $x or $y is true
xor	Xor	$x xor $y	True if either $x or $y is true, but not both
&&	And	$x && $y	True if both $x and $y are true
||	Or	$x || $y	True if either $x or $y is true
!	Not	!$x	True if $x is not true

PHP Array & String Operators

Operator	Name	Example	Result
.	Concatenation	$txt1 . $txt2	Concatenation of $txt1 and $txt2
.=	Concatenation assignment	$txt1 .= $txt2	Appends $txt2 to $txt1

Operator	Name	Example	Result
+	Union	$x + $y	Union of $x and $y
==	Equality	$x == $y	Returns true if $x and $y have the same key/value pairs
===	Identity	$x === $y	Returns true if $x and $y have the same key/value pairs in the same order and of the same types
!=	Inequality	$x != $y	Returns true if $x is not equal to $y
<>	Inequality	$x <> $y	Returns true if $x is not equal to $y
!==	Non-identity	$x !== $y	Returns true if $x is not identical to $y

Program Control: PHP Conditional Statements

- Conditional statements are used to perform different actions based on different conditions.
- In PHP we have the following conditional statements:
  - **if statement:** executes some code if one condition is true
  - **if...else statement:** executes some code if a condition is true and another code if that condition is false
  - **if...elseif...else statement:** executes different codes for more than two conditions
  - **switch statement:** selects one of many blocks of code to be executed

PHP - The if Statement

- The syntax for conditionals in PHP is almost identical to that of JavaScript
- The if statement executes some code if one condition is true.
- Syntax
  - ```
    if (condition) {
      code to be executed if condition is true;
    }
    ```
  - ```
    <?php
    $t = date("H");

    if ($t < "20") {
      echo "Have a good day!";
    }
    ?>
    ```

PHP - The if...else Statement

- The **if...else statement** executes some code if a condition is true and another code if that condition is false.
- Syntax
  - ```
    if (condition) {
      code to be executed if condition is true;
    } else {
      code to be executed if condition is false;
    }
    ```

```php
<?php
$t = date("H");

if ($t < "20") {
  echo "Have a good day!";
} else {
  echo "Have a good night!";
}
?>
```
```

PHP - The if...elseif...else Statement

- The **if...elseif...else statement** executes different codes for more than two conditions.
- Syntax
  - ```
    if (condition) {
      code to be executed if this condition is true;
    } elseif (condition) {
      code to be executed if first condition is false and this condition is true;
    } else {
      code to be executed if all conditions are false;
    }
    ```
    ```

```php
// if statement with condition
if ( $hourOfDay > 6 && $hourOfDay < 12 ) {
   $greeting = "Good Morning";
}
else if ($hourOfDay == 12) {   // optional else if
   $greeting = "Good Noon Time";
}
else {                         // optional else branch
   $greeting = "Good Afternoon or Evening";
}
```

LISTING 8.7 Conditional statement using if . . . else

```php
<?php if ($userStatus == "loggedin") {  ?>
   <a href="account.php">Account</a>
   <a href="logout.php">Logout</a>
<?php }  else { ?>
   <a href="login.php">Login</a>
   <a href="register.php">Register</a>
<?php } ?>


<?php
   // equivalent to the above conditional
   if ($userStatus == "loggedin") {
      echo '<a href="account.php">Account</a> ';
      echo '<a href="logout.php">Logout</a>';
   }
   else {
      echo '<a href="login.php">Login</a> ';
      echo '<a href="register.php">Register</a>';
   }
?>
```

LISTING 8.8 Combining PHP and HTML in the same script

PHP switch Statement

- Use the **switch statement** to select one of many blocks of code to be executed.
- Syntax
  - ```
    switch (n) {
      case label1:
        code to be executed if n=label1;
        break;
      case label2:
        code to be executed if n=label2;
        break;
      case label3:
        code to be executed if n=label3;
        break;
        ...
      default:
        code to be executed if n is different from all labels;
    }
    ```
    ```

```html
<!DOCTYPE html>
<html>
<body>

<?php
$favcolor = "red";

switch ($favcolor) {
  case "red":
    echo "Your favorite color is red!";
    break;
  case "blue":
    echo "Your favorite color is blue!";
    break;
  case "green":
    echo "Your favorite color is green!";
    break;
  default:
    echo "Your favorite color is neither red, blue, nor green!";
}
?>

</body>
</html>
```

```php
switch ($artType) {
    case "PT":
        $output = "Painting";
        break;
    case "SC":
        $output = "Sculpture";
        break;
    default:
        $output = "Other";
}

// equivalent
if ($artType == "PT")
    $output = "Painting";
else if ($artType == "SC")
    $output = "Sculpture";
else
    $output = "Other";
```

LISTING 8.9 Conditional statement using switch

PHP Loops

- Loops are used to execute the same block of code again and again, as long as a certain condition is true.
- In PHP, we have the following loop types:
  - **while:** loops through a block of code as long as the specified condition is true
  - **do...while:** loops through a block of code once, and then repeats the loop as long as the specified condition is true
  - **for:** loops through a block of code a specified number of times
  - **foreach:** loops through a block of code for each element in an array

PHP while Loop

- The **while loop** - Loops through a block of code as long as the specified condition is true.
- Syntax
  - ```
    while (condition is true) {
      code to be executed;
    }
    ```

```php
<?php
$x = 1;

while($x <= 5) {
  echo "The number is: $x <br>";
  $x++;
}
?>
```
```

PHP do while Loop

- The **do...while loop** will always execute the block of code once, it will then check the condition, and repeat the loop while the specified condition is true.
- **In a do...while loop the condition is tested AFTER executing the statements within the loop.**
- Syntax
  - ```
    do {
      code to be executed;
    } while (condition is true);
    ```

```php
<?php
$x = 1;

do {
  echo "The number is: $x <br>";
  $x++;
} while ($x <= 5);
?>
```
```

```php
$count = 0;
while ($count < 10)
{
    echo $count;
    $count++;
}

$count = 0;
do
{
    echo $count;
    $count++;
} while ($count < 10);
```

LISTING 8.10 while loops

PHP for Loop

- The **for loop** - Loops through a block of code a specified number of times.
  - It is used when you know in advance how many times the script should run.
- Syntax
  - ```
    for (init counter; test counter; increment counter) {
      code to be executed for each iteration;
    }
    ```

```php
<?php
for ($x = 0; $x <= 10; $x++) {
  echo "The number is: $x <br>";
}
?>
```
```

PHP foreach Loop

- The **foreach loop** - Loops through a block of code for each **element in an array**.
  - Works only on arrays, and is used to loop through each key/value pair in an array.
- For every loop iteration, the value of the current array element is assigned to $value and the array pointer is moved by one, until it reaches the last array element.
- Syntax
  - ```
    foreach ($array as $value) {
      code to be executed;
    }
    ```

```php
<?php
$colors = array("red", "green", "blue", "yellow");

foreach ($colors as $value) {
  echo "$value <br>";
}
?>
```
```

PHP Arrays

- An array is a special variable, which can hold more than one value at a time.
  - e.g. If you have a list of items (a list of car names)
- In PHP, the **array()** function is used to create an array
  - `array();`
- In PHP, there are three types of arrays:
  - **Indexed arrays:** Arrays with a numeric index
  - **Associative arrays:** Arrays with named keys
  - **Multidimensional arrays:** Arrays containing one or more arrays

```php
<?php
$cars = array("Volvo", "BMW", "Toyota");
echo "I like " . $cars[0] . ", " .
$cars[1] . " and " . $cars[2] . ".";
?>
```

https://www.w3schools.com/php/php_ref_array.asp

```
<!DOCTYPE html>
<html>
<body>

<?php
$cars = array("Volvo", "BMW", "Toyota");
$arrlength = count($cars);

for($x = 0; $x < $arrlength; $x++) {
  echo $cars[$x];
  echo "<br>";
}
?>

</body>
</html>
```

PHP Functions

- **Q-Recap:**
  - A function is a block of statements that can be used repeatedly in a program.
  - A function will not execute automatically when a page loads.
  - A function will be executed by a call to the function.
- Just like with JavaScript, **a function** in PHP contains a small bit of code that accomplishes one thing.
  - PHP has over 1000 built-in functions that can be called directly, from within a script, to perform a specific task.
  - The real power of PHP comes from its functions.
- These functions can be made to behave differently based on the values of their parameters

User Defined Function in PHP

- A user-defined function declaration starts with the word function.
- A function name must start with a letter or an underscore.
  - Function names are NOT case-sensitive.
  - Give the function a name that reflects what the function does!
- Syntax
  - ```
    function functionName() {
      code to be executed;
    }
    ```

```php
<?php
function writeMsg() {
  echo "Hello world!";
}

writeMsg(); // call the function
?>
```
```

PHP Function Arguments

- Information can be passed to functions through arguments.
  - An argument is just like a variable.
- Arguments are specified after the function name, inside the parentheses.
- You can add as many arguments as you want, just separate them with a comma.
- By default, arguments passed to functions are **passed by value** in PHP.
  - This means that PHP passes a copy of the variable so if the parameter is modified within the function, it does not change the original.
- PHP also allows arguments to functions to be **passed by reference**, which will allow a function to change the contents of a passed variable.
  - A parameter passed by reference points the local variable to the same place as the original, so if the function changes it, the original variable is changed as well.

```
<!DOCTYPE html>
<html>
<body>

<?php
function familyName($fname, $year) {
  echo "$fname Refsnes. Born in $year <br>";
}

familyName("Hege","1975");
familyName("Stale","1978");
familyName("Kai Jim","1983");
?>

</body>
</html>
```

Reading Task

- **Strict vs no-strict declarations in functions, Functions - Returning values, Return Type Declarations, Passing Arguments by Reference, PHP Global Variables – Superglobals, PHP Regular Expressions, array methods**
  - **Source: Chapter 8 & 9 plus online resources**